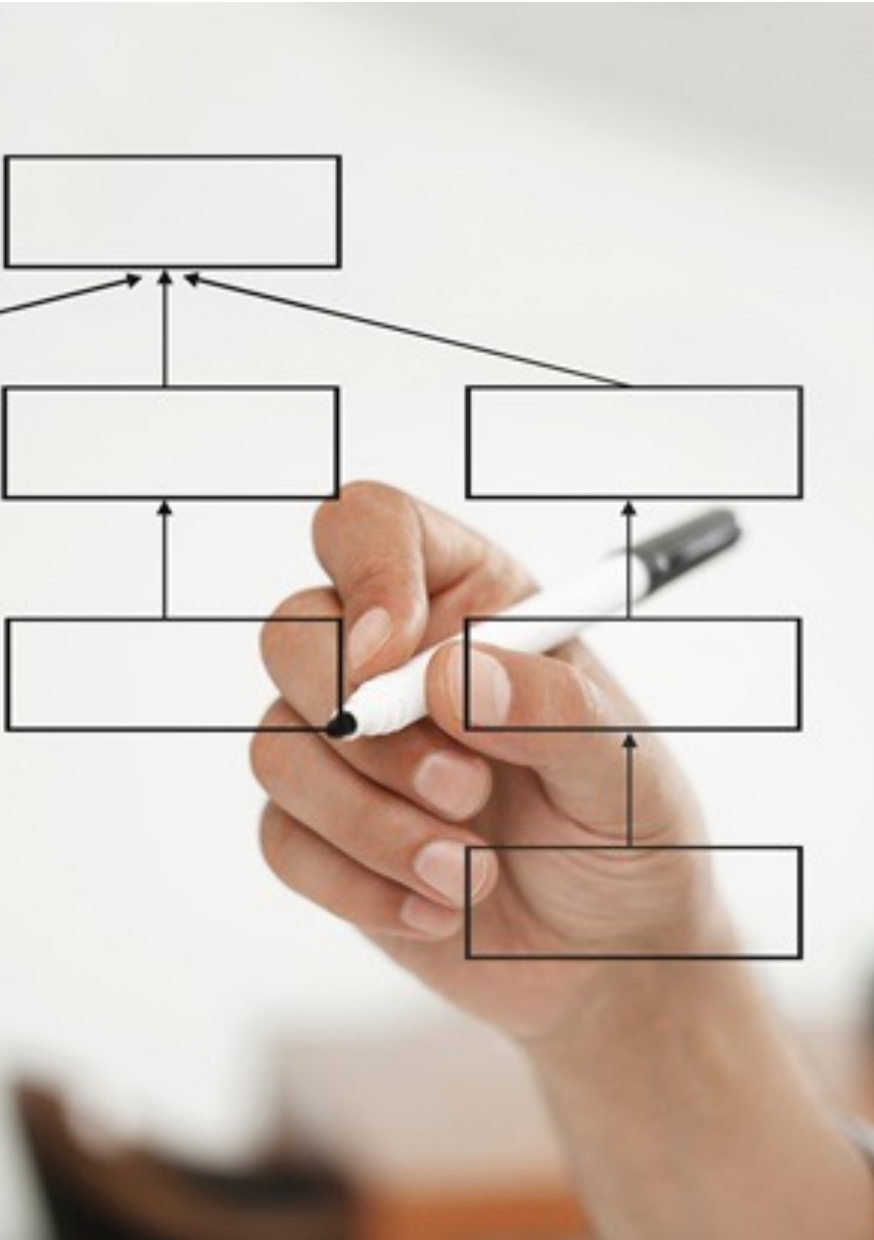




Estrutura de Dados I

CÁSSIO CAPUCHO PEÇANHA - 02

Comando if

- A forma geral de um comando if é:

```
if (condição) {  
    sequência de comandos;  
}
```

- A expressão, na condição, será avaliada:
 - Se ela for zero (falsa), a declaração não será executada;
 - Se a condição for diferente de zero (verdadeira) a declaração será executada.

Exemplo if

```
#include <stdio.h>
#include <stdlib.h>
int main(){
    int num;
    printf("Digite um numero: ");
    scanf("%d",&num);

    if(num > 10){
        printf("O numero eh maior do que 10.\n");
    }

    return 0;
}
```

Condição do if

- A condição pode ser uma expressão usando operadores matemáticos, lógicos e relacionais
 - +, -, *, /, %
 - &&, ||
 - >, <, >=, <=, ==, !=
- Ex:
 - (x > 10 && y <= x-1)

Comando else

- O comando if-else tem a seguinte forma geral:

```
if(condição) {  
    sequência de comandos 1;  
  
} else{  
    sequência de comandos 2;  
  
}
```

Comando else

- A expressão da condição será avaliada:
 - Se ela for diferente de zero (verdadeiro), a seqüência de comandos 1 será executada.
 - Se for zero (falso) a seqüência de comandos 2 será executada.
- Note que quando usamos a estrutura if-else, uma das duas declarações será executada.
- Não há obrigatoriedade em usar o else

O comando switch

- O comando switch é próprio para se testar uma variável em relação a diversos valores pré-estabelecidos.
 - Parecido com if-else-if, porém não aceita expressões, **apenas constantes**.
 - O switch testa a variável e executa a declaração cujo “case” corresponda ao valor atual da variável.

O comando switch

- Forma geral do comando switch

```
switch (expressão) {  
    case valor 1:  
        sequência de comandos 1;  
        break;  
    case valor k:  
        sequência de comandos k;  
        break;  
    ...  
    default:  
        sequência de comandos padrão;  
        break;  
}
```


O comando switch

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    char ch;
    printf("Digite um simbolo de pontuacao: ");
    ch = getchar();
    switch( ch ) {
        case '.':
            printf("Ponto.\n"); break;
        case ',':
            printf("Virgula.\n"); break;
        case ':':
            printf("Dois pontos.\n"); break;
        case ';':
            printf("Ponto e virgula.\n"); break;
        default :
            printf("Nao eh pontuacao.\n");
    }

    return 0;
}
```

Comando while

- Equivale ao comando “enquanto” utilizado nos pseudo-códigos.
 - Repete a sequência de comandos enquanto a condição for verdadeira.
 - Repetição com Teste no Início
- Esse comando possui a seguinte forma geral:

```
while (condição) {  
    sequência de comandos;  
}
```

Comando while - exemplo

- Faça um programa que mostra na tela os número de 1 a 100

```
int main() {  
    // programa que mostra na tela números de 1 ate 100  
    printf(" 1 2 3 4 .... ");  
    return 0;  
}
```

- A solução acima é inviável para valores grandes. Precisamos de algo mais eficiente e inteligente

Comando while - exemplo

- Faça um programa que mostra na tela os número de 1 a 100

```
int main() {  
    // programa que mostra na tela números de 1 ate 100  
    int numero;  
    numero = 1; Inicializa o contador  
    while(numero <= 100) {  
        printf("%d", numero);  
        numero = numero + 1; Incrementa o contador  
    }  
    return 0;  
}
```

- Observe que a variável **numero** é usada como um **contador**, ou seja, vai contar quantas vezes o loop será executado

Comando do-while

- Comando **while**: é utilizado para repetir um conjunto de comandos zero ou mais vezes.
 - Repetição com Teste no Início
- Comando **do-while**: é utilizado sempre que o bloco de comandos deve ser executado ao menos uma vez.
 - Repetição com Teste no Final

Comando do-while

- executa comandos
- avalia condição:
 - se verdadeiro, re-executa bloco de comandos
 - caso contrário, termina o laço
- Sua forma geral é (sempre termina com ponto e vírgula!)

```
do {  
    sequência de comandos;  
} while (condição);
```

Comando do-while

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int i;
    do{
        printf("Escolha uma opcao:\n");
        printf("(1) Opcao 1\n");
        printf("(2) Opcao 2\n");
        printf("(3) Opcao 3\n");
        scanf("%d",&i);

    }while((i < 1) || (i > 3));

    system("pause");
    return 0;
}
```

Comando for

- O loop ou laço **for** é usado para repetir um comando, ou bloco de comandos, diversas vezes
 - Maior controle sobre o loop.
- Sua forma geral é:

```
for (inicialização; condição; incremento) {  
    sequência de comandos;  
}
```


Comando for

1. **inicialização**: iniciar variáveis (contador).
2. **condição**: avalia a condição. Se verdadeiro, executa comandos do bloco, senão encerra laço.
3. **incremento**: ao término do bloco de comandos, incrementa o valor do contador
4. repete o processo até que a **condição** seja falsa.

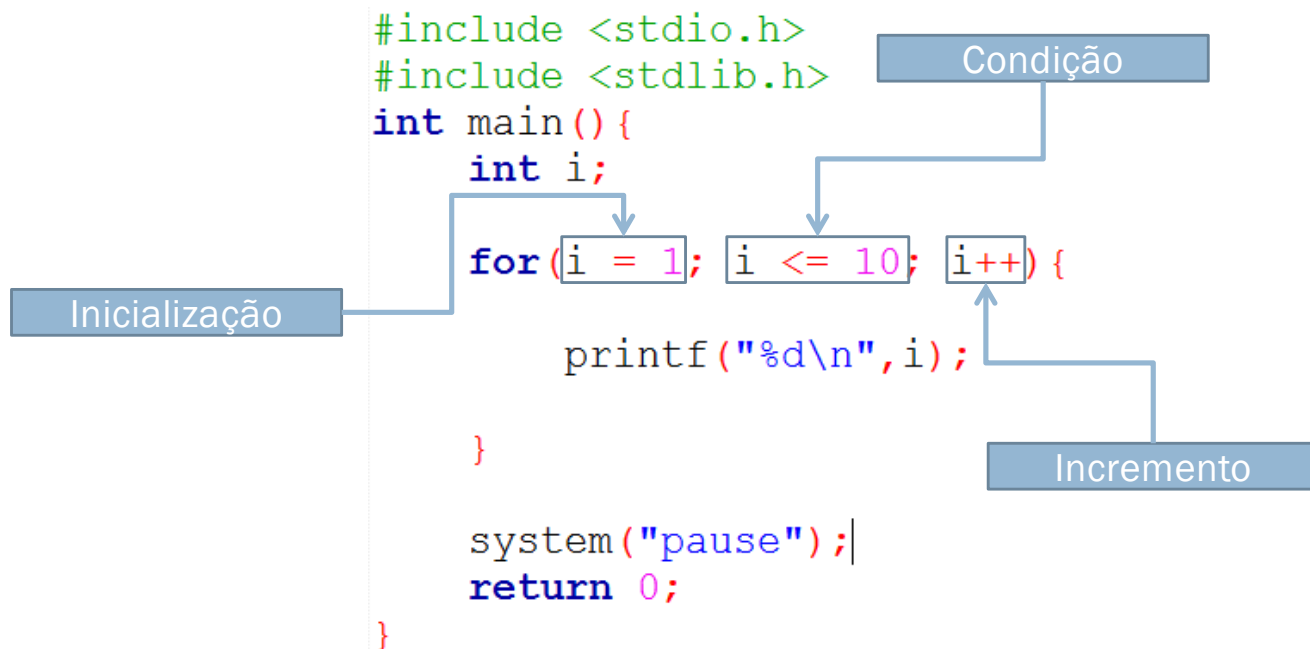
```
for(inicialização; condição; incremento){  
    sequência de comandos;  
}
```

Comando for

- Em geral, utilizamos o comando **for** quando precisamos ir de um valor inicial até um valor final.
- Para tanto, utilizamos uma variável para a realizar a contagem
 - Exemplo: `int i;`
- Nas etapas do comando **for**
 - Inicialização: atribuímos o valor inicial a variável
 - Condição: especifica a condição para continuar no *loop*
 - Exemplo: seu valor final
 - Incremento: atualiza o valor da variável usada na contagem

Comando for

- Exemplo: imprime os valores de 1 até 10



Por que usar array?

As variáveis declaradas até agora são capazes de armazenar um único valor por vez.

- Sempre que tentamos armazenar um novo valor dentro de uma variável, o valor antigo é sobrescrito e, portanto, perdido

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    float x = 10;
    printf("x = %f\n", x);
    x = 20;
    printf("x = %f\n", x);
    system("pause");
    return 0;
}
```

Saída

x = 10.000000

x = 20.000000

Array

- Array ou “vetor” é a forma mais familiar de dados estruturados.
- Basicamente, um array é uma sequência de elementos do mesmo tipo, onde cada elemento é identificado por um índice
 - A idéia de um array ou “vetor” é bastante simples: criar um conjunto de variáveis do mesmo tipo utilizando apenas um nome.

Array - Problema

- Imagine o seguinte problema
 - leia as notas de uma turma de cinco estudantes e depois imprima as notas que são maiores do que a média da turma.
- Um algoritmo para esse problema poderia ser o mostrado a seguir.

Array - Solução

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    float n1,n2,n3,n4,n5;
    printf("Digite a nota de 5 estudantes: ");
    scanf("%f",&n1);
    scanf("%f",&n2);
    scanf("%f",&n3);
    scanf("%f",&n4);
    scanf("%f",&n5);
    float media = (n1+n2+n3+n4+n5)/5.0;
    if(n1 > media) printf("nota: %f\n",n1);
    if(n2 > media) printf("nota: %f\n",n2);
    if(n3 > media) printf("nota: %f\n",n3);
    if(n4 > media) printf("nota: %f\n",n4);
    if(n5 > media) printf("nota: %f\n",n5);

    return 0;
}
```

Array

O algoritmo anterior apresenta uma solução possível para o problema apresentado

Porém, essa solução é inviável para grandes quantidades de alunos

- Imagine se tivéssemos de processar as notas de 100 alunos

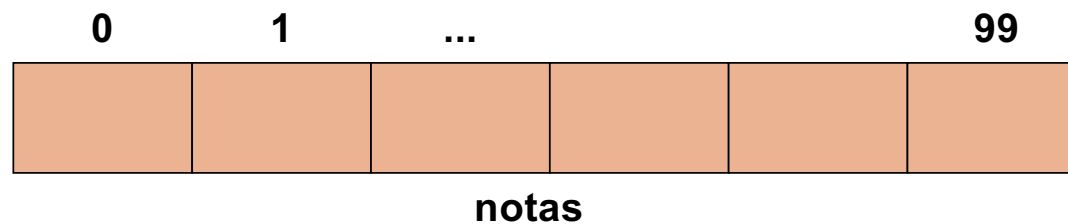
Array

Para 100 alunos, precisamos de:

- Uma variável para armazenar a nota de cada aluno
 - 100 variáveis
- Um comando de leitura para cada nota
 - 100 scanf()
- Um somatório de **100 notas**
- Um comando de teste para cada aluno
 - 100 comandos if.
- Um comando de impressão na tela para cada aluno
 - 100 printf()

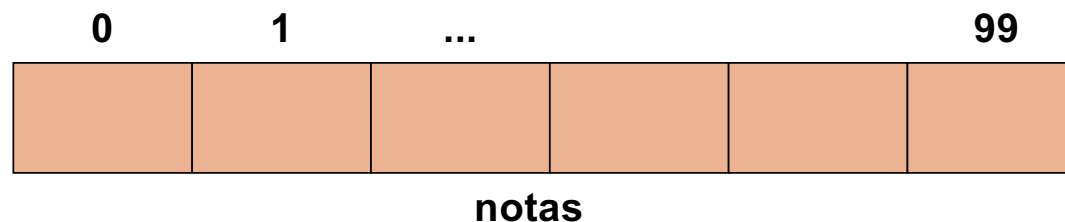
Array - Definição

- As variáveis têm relação entre si
 - todas armazenam notas de alunos
- Podemos declará-las usando um ÚNICO nome para todos os 100 alunos
 - notas: conjunto de 100 valores acessados por um índice
 - Isso é um **array**!



Array - Declaração

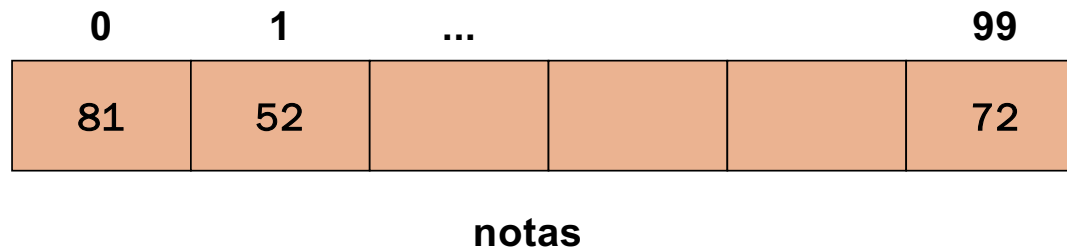
- Arrays são agrupamentos de dados adjacentes na memória. Declaração:
 - *tipo_dado nome_array[tamanho];*
- O comando acima define um array de nome **nome_array**, capaz de armazenar **tamanho** elementos adjacentes na memória do tipo **tipo_dado**
 - Ex: `int notas[100];`



Array - Declaração

- Em um array, os elementos são acessados especificando o índice desejado entre **colchetes** `[]`
- A numeração começa sempre do zero
- Isto significa que um array de 100 elementos terá índices de 0 a 99:
 - `notas[0]`, `notas[1]`, `notas[2]`, ..., `notas[99]`

```
int notas[100];  
notas[0] = 81;  
notas[1] = 55;  
...  
notas[99] = 72;
```



Array - Definição

■ Observação

- Se o usuário digitar mais de 100 elementos em um array de 100 elementos, o programa tentará ler normalmente.
- Porém, o programa os armazenará em uma parte não reservada de memória, pois o espaço reservado para o array foi para somente 100 elementos.
- Isto pode resultar nos mais variados erros durante a execução do programa.

Array = variável

- Cada elemento do array tem todas as características de uma variável e pode aparecer em expressões e atribuições (respeitando os seus tipos)
 - `notas[2] = x + notas[3];`
 - `if (notas[2] > 60)`
- Ex: somar todos os elementos de notas:

```
int soma = 0;  
for(i=0; i < 100; i++)  
    soma = soma + notas[i];
```

Percorrendo um array

- Podemos usar um comando de repetição (for, while e do-while) para percorrer um array
- Exemplo: somando os elementos de um array de 5 elementos

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int lista[5] = {3, 51, 18, 2, 45};
    int i, soma = 0;
    for(i = 0; i < 5; i++)
        soma = soma + lista[i];

    printf("soma = %d\n", soma);

    return 0;
}
```

Variáveis		
soma	i	lista[i]
0		
3	0	3
54	1	51
72	2	18
74	3	2
119	4	45
	5	

Array - Características

- Características básicas de um Array
 - Estrutura homogênea, isto é, é formado por elementos do mesmo tipo.
 - Todos os elementos da estrutura são igualmente acessíveis, isto é, o tempo e o tipo de procedimento para acessar qualquer um dos elementos do array são iguais.
 - Cada elemento do array tem um índice próprio segundo sua posição no conjunto

Array - Problema

- Voltando ao problema anterior
 - leia as notas de uma turma de cinco estudantes e depois imprima as notas que são maiores do que a média da turma.

Array - Solução

- Um algoritmo para esse problema usando array:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    float notas[5];
    int i;
    printf("Digite as notas dos estudantes\n");
    for(i = 0; i < 5; i++){
        printf("Nota do estudante %d:", i);
        scanf("%f", &notas[i]);
    }
    float media = 0;
    for(i = 0; i < 5; i++)
        media = media + notas[i];
    media = media / 5;

    for(i = 0; i < 5; i++)
        if(notas[i] > media)
            printf("Notas: %f\n", notas[i]);

    return 0;
}
```

Variáveis

- As variáveis vistas até agora podem ser classificados em duas categorias:
 - simples: definidas por tipos **int**, **float**, **double** e **char**;
 - compostas homogêneas (ou seja, do mesmo tipo): definidas por **array**.
- No entanto, a linguagem C permite que se criem novas estruturas a partir dos tipos básicos.
 - **struct**

Estruturas

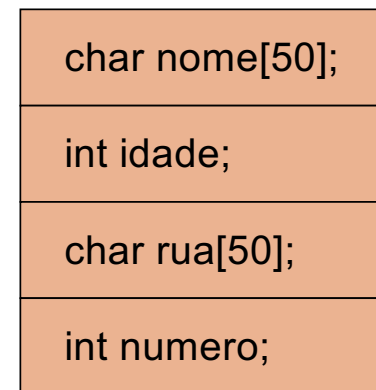
- Uma estrutura pode ser vista como um **novo tipo de dado**, que é formado por composição de variáveis de outros tipos
 - Pode ser declarada em qualquer escopo.
 - Ela é declarada da seguinte forma:

```
struct nomestruct{  
    tipo1 campo1;  
    tipo2 campo2;  
    ...  
    tipoN campoN;  
};
```

Estruturas

- Uma estrutura pode ser vista como um agrupamento de dados.
- Ex.: cadastro de pessoas.
 - Todas essas informações são da mesma pessoa, logo podemos agrupá-las.
 - Isso facilita também lidar com dados de outras pessoas no mesmo programa

```
struct cadastro{  
    char nome[50];  
    int idade;  
    char rua[50]  
    int numero;  
};
```



cadastro

Estruturas - declaração

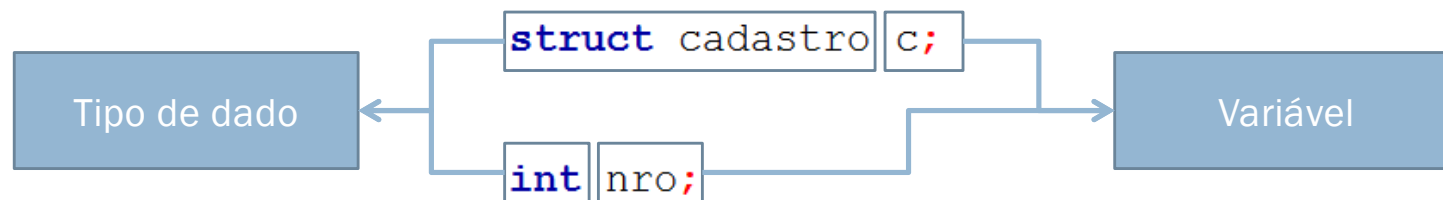
- Uma vez definida a estrutura, uma **variável** pode ser declarada de modo similar aos tipos já existente:

```
struct cadastro c;
```

- Obs: por ser um tipo definido pelo programador, usa-se a palavra **struct** antes do tipo da nova variável

Estruturas - declaração

Obs: por ser um tipo definido pelo programador, usa-se a palavra **struct** antes do tipo da nova variável



Acesso às variáveis

Como é feito o acesso às variáveis da estrutura?

- Cada variável da estrutura pode ser acessada com o operador ponto “.”.
- Ex.:

```
//declarando a variável
struct cadastro c;

//acessando os seus campos
strcpy(c.nome, "João");
scanf("%d", &c.idade);
strcpy(c.rua, "Avenida 1");
c.numero = 1082;
```


Comando typedef

- A linguagem C permite que o programador defina os seus próprios tipos com base em outros tipos de dados existentes.
- Para isso, utiliza-se o comando ***typedef***, cuja forma geral é:
 - **typedef tipo_existente novo_nome;**

Comando typedef

■ Exemplo

- Note que o comando **typedef** não cria um novo tipo chamado **inteiro**. Ele apenas cria um sinônimo (**inteiro**) para o tipo **int**

```
#include <stdio.h>
#include <stdlib.h>

typedef int inteiro;

int main() {
    int x = 10;
    inteiro y = 20;
    y = y + x;
    printf("Soma = %d\n", y);

    return 0;
}
```

Comando typedef

- O **typedef** é muito utilizado para definir nomes mais simples para estrutura, evitando carregar a palavra **struct** sempre que referenciamos a estrutura

```
struct cadastro{
    char nome[300];
    int idade;
};
// redefinindo o tipo struct cadastro
typedef struct cadastro CadAlunos;

int main(){
    struct cadastro aluno1;
    CadAlunos aluno2;

    return 0;
}
```